Tutorial 12: Basic AutoEncoders

Laiba Niazi

536672

1. Objective

The primary goal of this experiment is to design, implement, and evaluate the reconstruction efficiency of a standard convolutional autoencoder versus an improved version utilizing strided and transposed convolutions. Both models were tested using the MNIST dataset of handwritten digits.

2. Dataset and Preprocessing

- **Dataset:** The MNIST dataset was loaded and converted into PyTorch tensors**.**
- **Normalization:** Input images are grayscale28x28 pixels.
- **Loading:** Data loaders were configured with a batch size of 128.
- **Hardware:** The implementation detects and utilizes CUDA for GPU acceleration when available

3. Methodology & Architectures

3.1 Basic Autoencoder

The BasicAutoencoder utilizes a standard approach to feature extraction and spatial recovery:

- **Encoder:** Consists of two convolutional layers with 3x3 kernels followed by MaxPool2d layers. This reduces the spatial dimensions from 28x28 to a latent representation of 7x7.
- **Decoder:** Employs Upsample (bilinear/nearest) followed by standard convolutions to regain the original resolution. It uses a Sigmoid activation in the final layer to bound pixel values between [0, 1].

```python
# 1. Basic Autoencoder
class BasicAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2), # 14x14
            nn.Conv2d(16, 8, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2)  # 7x7 (Latent Space)
        )
        self.decoder = nn.Sequential(
            nn.Upsample(scale_factor=2), nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
            nn.Upsample(scale_factor=2), nn.Conv2d(16, 1, 3, padding=1), nn.Sigmoid()
        )
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

3.2 Improved Autoencoder

The ImprovedAutoencoder introduces more sophisticated deep learning techniques:

- **Encoder:** Replaces Max Pooling with **Strided Convolutions** (stride=2), which allows the network to learn its own downsampling parameters. It also uses LeakyReLU to prevent the "dying ReLU" problem.
- **Decoder:** Utilizes **Transposed Convolutions** (ConvTranspose2d) instead of simple upsampling. This allows the decoder to learn the optimal way to reconstruct spatial features from the latent space.

```python
# 2. Improved Autoencoder (Task: Deeper layers + ConvTranspose)
class ImprovedAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride=2, padding=1), nn.LeakyReLU(0.2), # 14x14
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.LeakyReLU(0.2) # 7x7
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1, output_padding=1), nn.ReLU(),
            nn.ConvTranspose2d(32, 1, 3, stride=2, padding=1, output_padding=1), nn.Sigmoid()
        )
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

4. Training Configuration

Both models were trained under identical hyperparameters to ensure a fair comparison:

- **Loss Function:** Mean Squared Error (MSE), ideal for pixel-wise reconstruction tasks.
- **Optimizer:** Adam Optimizer (Learning Rate = 0.001).
- **Epochs:** 5.

```python
def train(model, loader, epochs=5):
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.MSELoss()
    loss_list = []
```

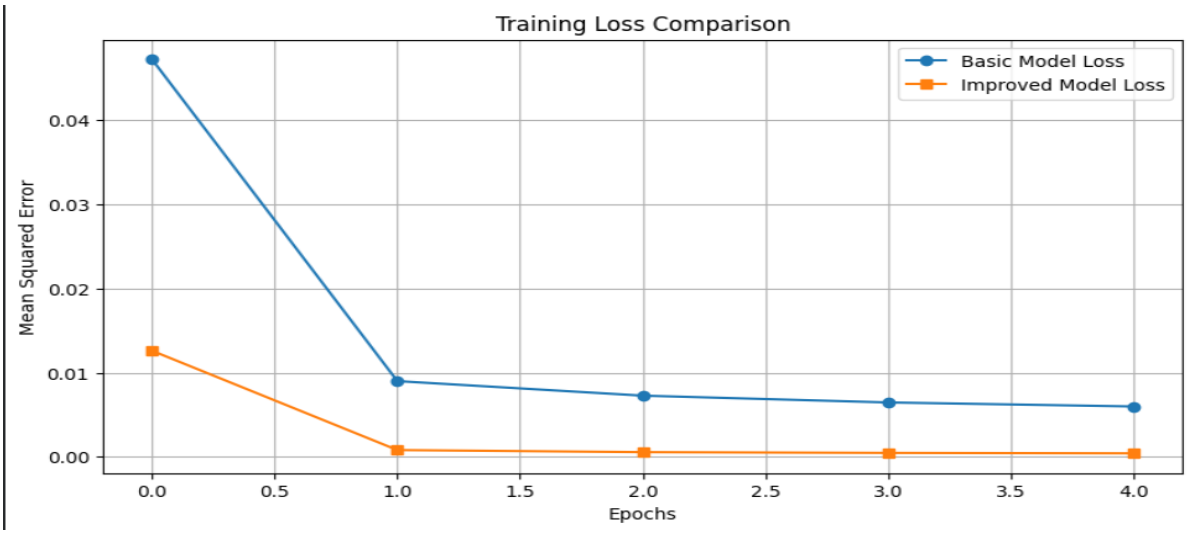
```
Training Basic Model...
Epoch 1 Loss: 0.0472
Epoch 2 Loss: 0.0090
Epoch 3 Loss: 0.0073
Epoch 4 Loss: 0.0065
Epoch 5 Loss: 0.0060

Training Improved Model...
Epoch 1 Loss: 0.0126
Epoch 2 Loss: 0.0008
Epoch 3 Loss: 0.0006
Epoch 4 Loss: 0.0005
Epoch 5 Loss: 0.0004
```

5. Performance Analysis

5.1 Training Loss Comparison

The loss curves demonstrate the convergence rate of both models.



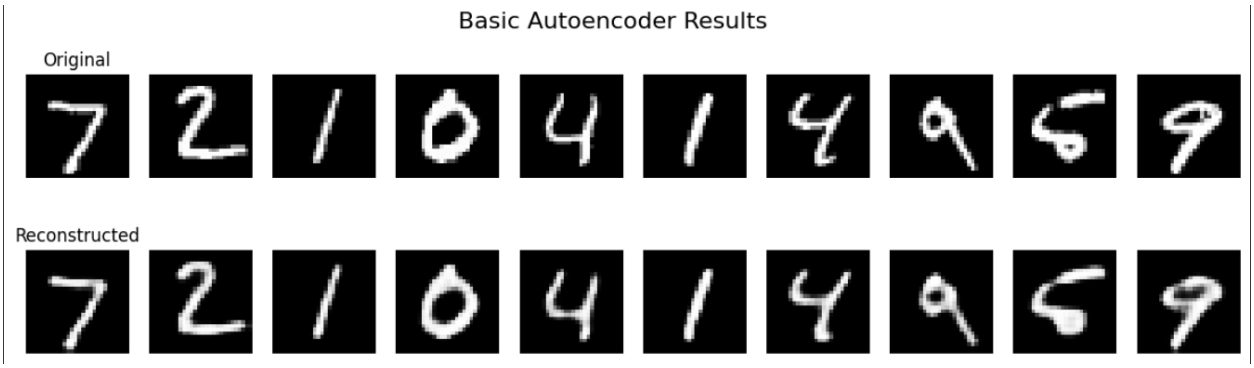
Observations:

- The **Improved Model** typically shows a significantly lower starting loss and faster convergence.
- The use of ConvTranspose2d helps the model reach a much lower Mean Squared Error compared to the Upsample method.

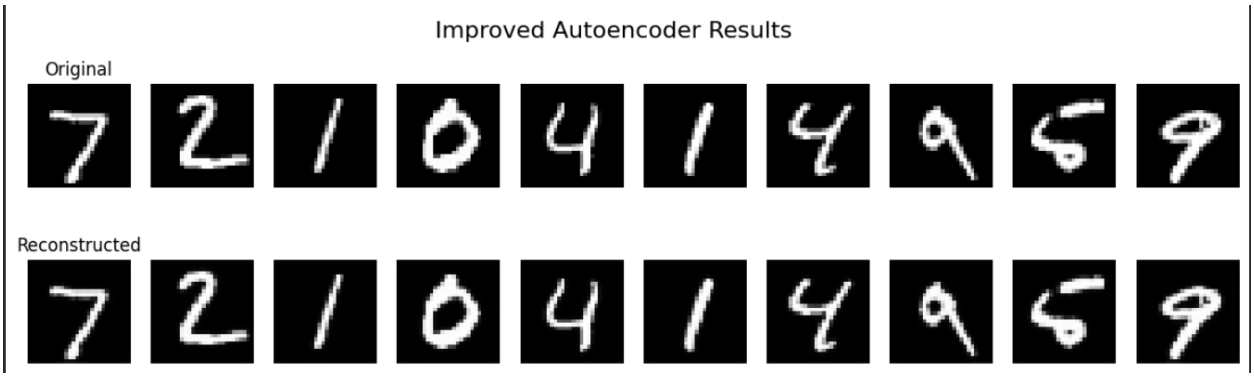
5.2 Visual Reconstruction Results

The qualitative performance is evaluated by passing test images through the bottleneck and comparing the output to the ground truth.

Basic Autoencoder Results



Improved Autoencoder Results



6. Discussion & Conclusion

The experiment confirms that architectural choices significantly impact the fidelity of reconstructed signals. While the Basic Autoencoder successfully captures the general shape of the digits, the **Improved Autoencoder** produces sharper, more accurate reconstructions. This is attributed to:

1. **Strided Convolutions:** Better feature retention during downsampling.
2. **ConvTranspose2d:** Learnable upsampling that restores high-frequency details.